Python Programming
Assignment No. 1

Name: Shraddha Surech Chavan
Roll Number: MS2502
GitHub Repository: shraddhachavan16/python

1 Variables

What is it: A variable is like a named box that stores a value. In Python, a variable is created when we give a value to a name.

Why do we need it: It stores data so that we can reuse it and change it whenever required.

Where and how is it used: We use variables for names, marks, calculations, user input, and many other values. A value is stored by using the `=` sign.

Example:

```
name = "Asha"
age = 20
marks = 85
```

Rules for variable names:

- A name can contain letters, digits, and underscore (_).
- It cannot start with a digit.
- It cannot contain spaces.
- Variable names are case-sensitive. Thus, `age` and `Age` are different.

2 Basic Data Types

A data type tells us what kind of value a variable contains.

2.1 Integer (`int`)

What is it: An integer is a whole number with no decimal point. It can be positive, negative, or zero.

Why do we need it: We need it for values that are counted as whole numbers.

Where and how is it used: It is useful for age, marks, quantity, and counting. We write the number directly without quotation marks.

```
age = 20
students = 40
temperature = -5
```

2.2 Float (`float`)

What is it: A float is a number that contains a decimal point.

Why do we need it: We need it for values that may not be whole numbers.

Where and how is it used: It is used for height, weight, percentage, price, and measurements. We write the value with a decimal point.

```
height = 1.65
percentage = 82.5
```

2.3 Boolean (`bool`)

What is it: A Boolean is a data type with only two possible values: `True` and `False`.

Why do we need it: We need it to show whether a condition is true or false.

Where and how is it used: It is mostly used in conditions and loops. We can write `True` or `False` directly, or get it after comparing two values.

```
is_passed = True
is_adult = age >= 18
```

2.4 String (`str`)

What is it: A string is any text or group of characters written inside quotation marks.

Why do we need it: We need it to save words, sentences, and other text in a program.

Where and how is it used: It is used for names, messages, addresses, and passwords. The text is placed inside single or double quotes.

```
name = "Shraddha"
course = 'Python Programming'
```

3 Arithmetic Operators

What are they: Arithmetic operators are symbols used to perform mathematical calculations.

Why do we need them: We need them to add, subtract, multiply, divide, and perform other calculations.

Where and how are they used: They are used for totals, averages, areas, and many calculations. We place an operator such as `+` or `*` between two numbers.

Operator	Meaning	Example
`+`	Addition	`10 + 3` gives `13`
`-`	Subtraction	`10 - 3` gives `7`
`*`	Multiplication	`10 * 3` gives `30`
`/`	Division	`10 / 2` gives `5.0`
`//`	Floor division	`10 // 3` gives `3`
`%`	Remainder	`10 % 3` gives `1`
`**`	Power	`2 ** 3` gives `8`

Example:

```
length = 8
width = 5
area = length * width
print(area)
```

Output: `40`

4 Comparison Operators

What are they: Comparison operators check the relation between two values. The answer is always `True` or `False`.

Why do we need them: We need them when a program has to check something and make a decision.

Where and how are they used: They are mostly used in `if` statements and loops. We put an operator such as `>` or `==` between the values.

Operator	Meaning	Example
`==`	Equal to	`5 == 5` is `True`
`!=`	Not equal to	`5 != 3` is `True`
`>`	Greater than	`5 > 3` is `True`
`<`	Less than	`5 < 3` is `False`
`>=`	Greater than or equal to	`5 >= 5` is `True`
`<=`	Less than or equal to	`3 <= 5` is `True`

```
marks = 72
print(marks >= 40)
```

Output: `True`

Important: `=` assigns a value, while `==` compares two values.

5 Assignment Operators

What are they: Assignment operators are symbols used to put a value into a variable or update its old value.

Why do we need them: We need them to store values and to update them in a short and easy way.

Where and how are they used: They are used whenever we create or change a variable. The `=` sign stores a value, while operators such as `+=` calculate and store together.

Operator	Example	Same as
`=`	`x = 5`	Assign `5` to `x`
`+=`	`x += 2`	`x = x + 2`
`-=`	`x -= 2`	`x = x - 2`
`*=`	`x *= 2`	`x = x * 2`
`/=`	`x /= 2`	`x = x / 2`
`//=`	`x //= 2`	`x = x // 2`
`%=`	`x %= 2`	`x = x % 2`
`**=`	`x **= 2`	`x = x ** 2`

Example:

```
x = 10
x += 5
print(x)
```

Output: `15`

6 Type Conversion

What is it: Type conversion means changing the type of a value, for example from a string to an integer.

Why do we need it: Sometimes two values have different types and cannot be used together. Conversion makes them suitable for the required work.

Where and how is it used: It is commonly used with user input because `input()` gives text. We use `int()`, `float()`, `str()`, or `bool()` to change the type.

Function	Purpose
`int()`	Converts a value to an integer
`float()`	Converts a value to a float
`str()`	Converts a value to a string
`bool()`	Converts a value to Boolean

Example:

```
age_text = "20"
age = int(age_text)
price = float("99.50")
number_text = str(100)
```

There are two types of conversion:

- **Implicit conversion:** Python converts the type automatically, such as `5 + 2.5` giving `7.5`.
- **Explicit conversion:** The programmer converts the type using functions such as `int()` or `float()`.

7 Input and Output

7.1 The `input()` Function

What is it: The `input()` function takes a value typed by the user. Python receives that value as a string.

Why do we need it: We need it when the program has to ask the user for some information.

Where and how is it used: It is used to take a name, age, marks, or any other value. We write a message inside `input()` and save the answer in a variable.

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
```

7.2 The `print()` Function

What is it: The `print()` function shows text or values on the screen.

Why do we need it: We need it to show messages and answers produced by the program.

Where and how is it used: It is used whenever output must be shown. We place the required message or variable inside `print()`.

```
print("Hello", name)
print("Age:", age)
```

`sep` changes the separator between values, and `end` changes how the printed line ends.

```
print("A", "B", sep="-")
print("Hello", end=" ")
print("Python")
```

8 Strings

What is a string: A string is a sequence of letters, numbers, spaces, or symbols written as text inside quotes.

Why do we need it: We need strings because programs often work with names, messages, and sentences.

Where and how is it used: Strings are used wherever text is needed. We make them by placing characters inside single, double, or triple quotes.

8.1 String Creation

What is it: String creation means making a text value in Python.

Why do we need it: We need it before we can save or work with any text.

Where and how is it used: It is used for all kinds of text. Write the characters inside single, double, or triple quotes.

```
name = 'Shraddha'
course = "Python"
message = """Welcome to
Python Programming"""
```

8.2 Indexing

What is it: Indexing means finding one character by using its position number in the string.

Why do we need it: We need it when only one particular character is required.

Where and how is it used: It is used while checking or processing text. Write the position inside square brackets, such as `word[0]`.

- The first character has index `0`.
- The last character has index `-1`.

```
word = "Python"
print(word[0])
print(word[-1])
```

Output:

```
P
n
```

8.3 Slicing

What is it: Slicing means taking a smaller part from a string.

Why do we need it: We need it to get a word part, selected characters, or a reversed string.

Where and how is it used: It is used while working with text. We write start, stop, and step positions inside square brackets. The stop position is not included.

```
word = "Python"
print(word[0:3])
print(word[2:])
print(word[:4])
print(word[::-1])
```

Output:

```
Pyt
thon
Pyth
nohtyP
```

8.4 String Operations

What are they: String operations are the different actions we can do on text, such as joining, repeating, and searching.

Why do we need them: We need them to change the form of text or use it in the required way.

Where and how are they used: They are used while preparing messages and processing text. We use operators such as `+` and methods such as `upper()`.

Operation	Purpose
`+`	Joins two strings
`*`	Repeats a string
`in`	Checks whether text is present
`len()`	Gives the number of characters
`upper()`	Converts letters to uppercase
`lower()`	Converts letters to lowercase

```
text = "Python"
print(text + " Programming")
print(text * 2)
print("Py" in text)
print(len(text))
```

Important: Strings are immutable. This means that individual characters of an existing string cannot be changed.

9 Lists

What is a list: A list is an ordered group of values stored in one variable. Its items can be changed.

Why do we need it: We need it when many related values have to be stored together.

Where and how is it used: Lists are used for marks, names, products, and similar collections. Values are separated by commas inside square brackets.

9.1 Creating Lists

What is it: Creating a list means placing a group of values inside square brackets.

Why do we need it: We need it to keep many items under one meaningful variable name.

Where and how is it used: It is used when we have a collection such as marks or names. Write the items with commas inside `[ ]`.

```
marks = [75, 82, 91, 68]
student = ["Asha", 21, 88.5]
empty_list = []
```

9.2 Indexing and Slicing

What are they: Indexing selects one item from a list, while slicing selects a part of the list.

Why do we need them: We need them to get only the required item or items.

Where and how are they used: They are used when reading list data. Write the position or range inside square brackets after the list name.

```
fruits = ["Apple", "Banana", "Mango", "Orange"]
print(fruits[0])
print(fruits[-1])
print(fruits[1:3])
```

Output:

```
Apple
Orange
['Banana', 'Mango']
```

9.3 Updating Elements

What is it: Updating a list means replacing an old item with a new value.

Why do we need it: We need it when stored information changes or needs correction.

Where and how is it used: It is used when list data must be changed. Give a new value to the required index. This is possible because lists are mutable.

```
numbers = [10, 20, 30]
numbers[1] = 200
print(numbers)
```

Output: `[10, 200, 30]`

9.4 Nested Lists

What is it: A nested list is simply a list kept as an item inside another list.

Why do we need it: We need it to arrange data in rows, columns, or separate groups.

Where and how is it used: It is useful for tables and matrices. The first index chooses the inner list, and the second index chooses an item from it.

```
matrix = [[1, 2], [3, 4], [5, 6]]
print(matrix[1])
print(matrix[1][0])
```

Output:

```
[3, 4]
3
```

9.5 Basic List Methods

What are they: List methods are ready-made Python commands that help us work with the items in a list.

Why do we need them: They let us add, remove, arrange, or count items without writing long programs.

How are they used: Write the list name, a dot, and the method name, such as `a.append(40)`. Write the `len()` function as `len(a)`.

1. `append()`

Syntax: `list.append(item)`

Use `append()` when a new item must be added at the end of a list. It changes the original list.

```
numbers = [10, 20, 30]
numbers.append(40)
print(numbers)
```

Output: `[10, 20, 30, 40]`

2. `insert()`

Syntax: `list.insert(index, item)`

Use `insert()` when an item must be added at a particular position. The first value is the index (position), and the second value is the new item. Index counting starts from `0`.

```
numbers = [10, 30]
numbers.insert(1, 20)
print(numbers)
```

Output: `[10, 20, 30]`

3. `remove()`

Syntax: `list.remove(value)`

Use `remove()` when you know the value that should be deleted. It removes the first matching value. If the value is not in the list, Python gives an error.

```
numbers = [10, 20, 30]
numbers.remove(20)
print(numbers)
```

Output: `[10, 30]`

4. `pop()`

Syntax: `list.pop(index)` or `list.pop()`

Use `pop()` when an item must be removed and saved for later use. If no index is given, it removes and returns the last item. An index can be given to remove an item from another position.

```
numbers = [10, 20, 30]
removed_item = numbers.pop()
print(numbers)
print(removed_item)
```

Output:

```
[10, 20]
30
```

5. `sort()`

Syntax: `list.sort()`

Use `sort()` to arrange the original list. Numbers are arranged from smallest to largest, and words are arranged alphabetically.

```
numbers = [30, 10, 20]
numbers.sort()
print(numbers)
```

Output: `[10, 20, 30]`

6. `reverse()`

Syntax: `list.reverse()`

Use `reverse()` to turn the current order of the list around. It does not sort the items; it only places them in the opposite order.

```
numbers = [10, 20, 30]
numbers.reverse()
print(numbers)
```

Output: `[30, 20, 10]`

7. `len()`

Syntax: `len(list)`

Use `len()` to find how many items are in a list. It is a built-in function, not a list method, so the list is written inside its parentheses.

```
numbers = [10, 20, 30]
print(len(numbers))
```

Output: `3`

Important difference: A list is mutable, so its elements can be changed. A string is immutable, so its characters cannot be changed directly.